

Trabalho Prático | DGT2816 Interação com sensores de smartphones e wearables Documentação do Trabalho Prático App Wear OS de Acessibilidade — Empresa Doma

Aluna: Alexandra Correa Bastos

Data: 21/06/2026

1. Introdução

Este documento descreve o desenvolvimento do Trabalho Prático da disciplina DGT2816 Interação com sensores de smartphones e wearables.

O trabalho consiste na criação de um aplicativo para Wear OS utilizando o Android Studio, focado em interagir com os recursos de hardware e gerenciamento de áudio do smartwatch para oferecer suporte de acessibilidade.

2. Contextualização

A empresa fictícia "Doma" possui o objetivo de melhorar a eficiência e a comunicação interna entre seus funcionários, com atenção especial aqueles com necessidades especiais. Foi proposto o desenvolvimento de um aplicativo Wear OS capaz de gerenciar ativamente as saídas de áudio do dispositivo. O intuito é garantir que o funcionário esteja sempre com um equipamento de áudio adequado (alto-falante ou fones Bluetooth) habilitado para ouvir informações críticas em tempo real, como leitura de notificações, instruções de treinamento e alertas de emergência.

3. Objetivos do Trabalho Prático

- Instalar e configurar o ambiente de desenvolvimento (Android Studio);
- Criar e configurar um emulador Wear OS (API 30);
- Desenvolver um aplicativo em Kotlin para Wear OS sem a dependência obrigatória de um smartphone conectado (standalone);
- Implementar a detecção em tempo real de saídas de áudio (TYPE_BUILTIN_SPEAKER e TYPE_BLUETOOTH_A2DP);
- Aplicar uma regra de negócio que força o redirecionamento do usuário para a tela de pareamento caso não haja fone Bluetooth conectado;
- Registrar e documentar a execução do aplicativo.

4. Tecnologias e Ferramentas Utilizadas

- **IDE:** Android Studio
- **Linguagem:** Kotlin

- **SDK:** Compilado na API 37 com suporte mínimo à API 30 (Android 11.0 "R")
- **Emulador:** Wear OS Small Round
- **APIs Nativas:** AudioManager, AudioDeviceCallback, Intent

5. Estrutura do Projeto

O projeto foi nomeado "ListaDeTarefas", com pacote `com.example.listadetarefas`, utilizando o template "No Activity" do Wear OS. A estrutura principal de arquivos focada na configuração e lógica do hardware ficou definida da seguinte forma:

ListaDeTarefas/

```
├─ app/
|   └─ build.gradle.kts
|       └─ src/main/
|           └─ AndroidManifest.xml
|           └─ java/com/example/listadetarefas/
|               └─ MainActivity.kt
```

6. Implementação

6.1 Permissões e Configurações Essenciais (AndroidManifest.xml)

Para permitir a interação com os recursos do sistema operativo do relógio, foram declaradas as permissões de sensores corporais (BODY_SENSORS) e de controle de tela (WAKE_LOCK). Além disso, o aplicativo foi categorizado explicitamente com a tag `com.google.android.wearable.standalone` definida como `true`, garantindo a compatibilidade exigida pelas versões mais recentes do ecossistema Wear OS.

6.2 Detecção de Saídas de Áudio (MainActivity.kt)

Toda a lógica de interação com o hardware foi centralizada na `MainActivity`. Utilizando a classe `AudioManager`, o aplicativo verifica se o smartwatch possui um alto-falante integrado (`TYPE_BUILTIN_SPEAKER`) e monitora se existe um fone de ouvido pareado ativo (`TYPE_BLUETOOTH_A2DP`).

6.3 Callback e Redirecionamento Automático (MainActivity.kt)

O sistema implementa o `registerAudioDeviceCallback`, que atua como um ouvinte para disparar avisos em tempo real ao plugar ou desplugar fones. Atendendo à premissa de acessibilidade da Doma, caso o aplicativo seja iniciado e não detecte nenhum fone Bluetooth, ele não trava nem exibe tela de erro; em vez disso, executa uma Intent com a ação `Settings.ACTION_BLUETOOTH_SETTINGS`. Isso leva o usuário diretamente e de forma automatizada à interface de pareamento do sistema.

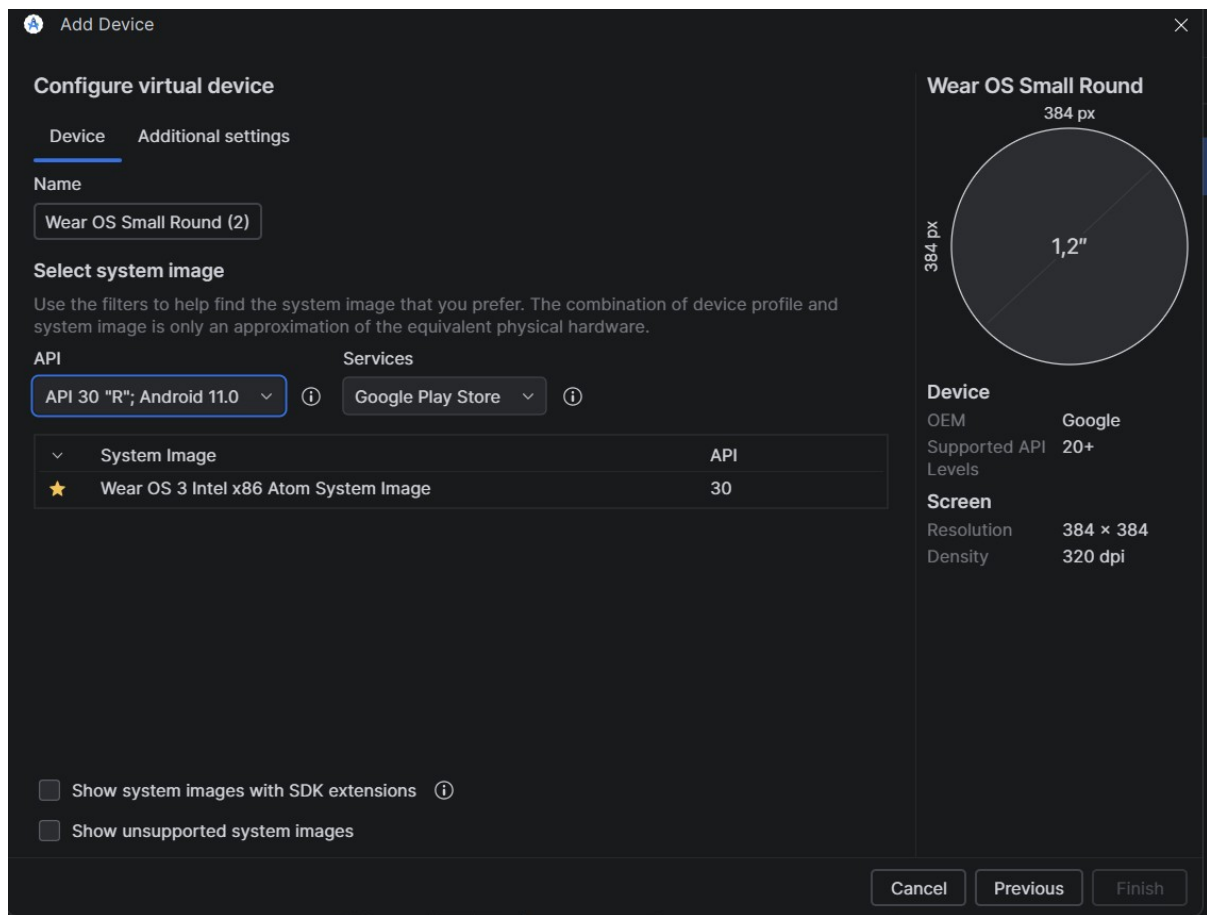
7. Como Executar o Projeto

1. Abrir o Android Studio e sincronizar o Gradle do projeto `ListaDeTarefas`.
2. Acessar o Device Manager e iniciar o emulador **Wear OS Small Round (API 30)**.
3. Executar o aplicativo através do botão *Run*.
4. O aplicativo inicializará e, ao detectar a ausência de um fone pareado no emulador, abrirá instantaneamente a tela de conexão Bluetooth do sistema Wear OS.

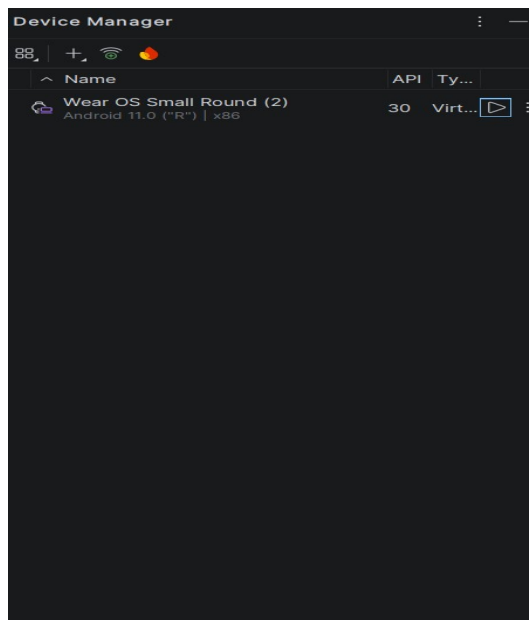
8. Capturas de Tela

As imagens a seguir evidenciam o passo a passo da configuração, a estruturação do código e o resultado final da execução do sistema no emulador.

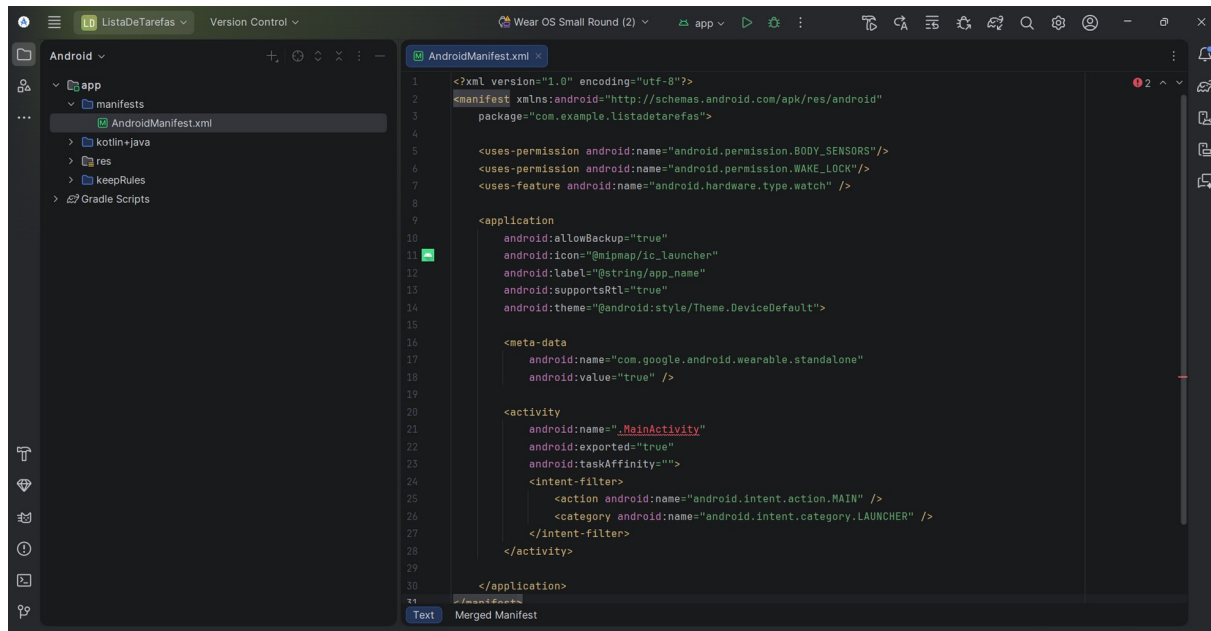
8.1 Configuração do Emulador Wear OS no Android Studio



8.2 Instância do Emulador no Device Manager



8.3 Configurações e Permissões no AndroidManifest



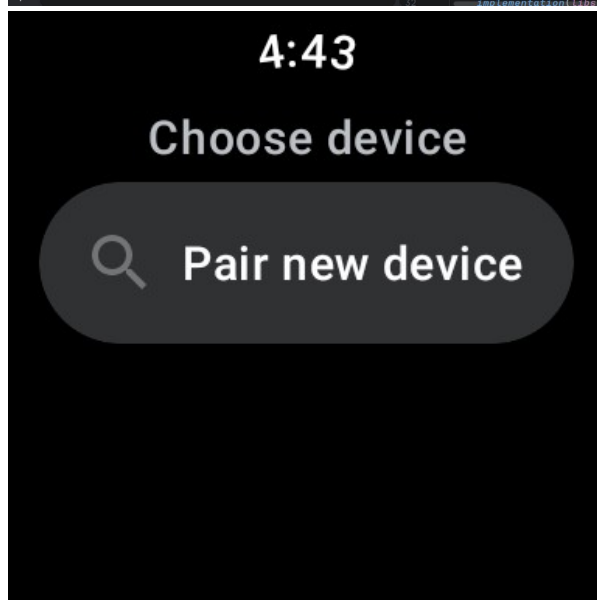
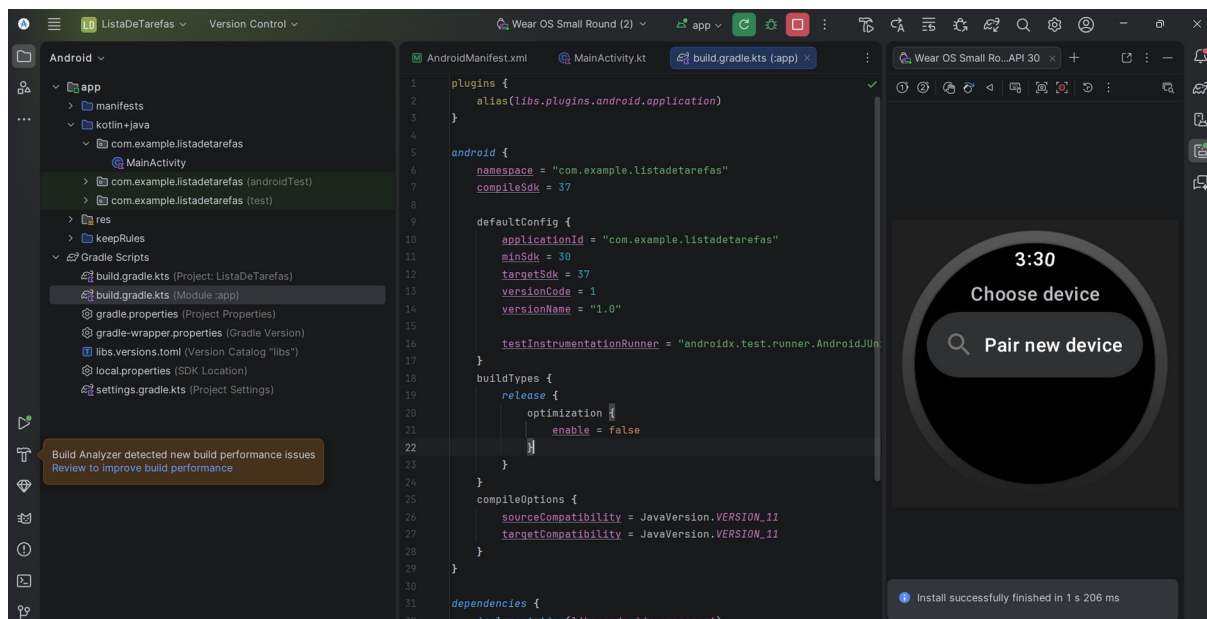
8.4 Lógica de Acessibilidade no Código-Fonte

```
AndroidManifest.xml x MainActivity.kt x
1 package com.example.listadetafezas
2
3 import android.app.Activity
4 import android.content.Context
5 import android.content.Intent
6 import android.content.pm.PackageManager
7 import android.media.AudioDeviceCallback
8 import android.media.AudioDeviceInfo
9 import android.media.AudioManager
10 import android.os.Bundle
11 import android.provider.Settings
12 import android.widget.Toast
13
14 3 Usages
15 class MainActivity : Activity() {
16     3 Usages
17     private lateinit var audioManager: AudioManager
18
19     override fun onCreate(savedInstanceState: Bundle?) {
20         super.onCreate(savedInstanceState)
21         // Aqui você vincularia o seu layout activity_main.xml se tivesse criado a interface gráfica completa
22         // setContentView(R.layout.activity_main)
23
24         audioManager = getSystemService(Context.AUDIO_SERVICE) as AudioManager
25
26         verificarDispositivosDeAudio()
27         registrarCallbackDeAudio()
28     }
29
30     4 Usages
```

```
AndroidManifest.xml x MainActivity.kt x
14 class MainActivity : Activity() {
15     22
16     private fun audioOutputAvailable(type: Int): Boolean {
17         if (!packageManager.hasSystemFeature(featureName = PackageManager.FEATURE_AUDIO_OUTPUT)) {
18             return false
19         }
20         return audioManager.getDevices(flags = AudioManager.GET_DEVICES_OUTPUTS).any { it.type == type }
21     }
22
23     1 Usage
24     private fun verificarDispositivosDeAudio() {
25         val isSpeakerAvailable = audioOutputAvailable(type = AudioDeviceInfo.TYPE_BUILTIN_SPEAKER)
26         val isBluetoothHeadsetConnected = audioOutputAvailable(type = AudioDeviceInfo.TYPE_BLUETOOTH_A2DP)
27
28         if (isSpeakerAvailable) {
29             Toast.makeText(context = this, text = "Alto-falante detectado. Sistema Doma pronto.", duration = Toast.LENGTH_SHORT)
30         }
31
32         if (!isBluetoothHeadsetConnected) {
33             solicitarConexaoBluetooth()
34         }
35     }
36
37     1 Usage
38     private fun solicitarConexaoBluetooth() {
39         Toast.makeText(context = this, text = "Por favor, conecte um fone Bluetooth para melhor acessibilidade.", duration = Toast.LENGTH_SHORT)
40         val intent = Intent(action = Settings.ACTION_BLUETOOTH_SETTINGS).apply {
41             addFlags(Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK)
42             putExtra(name = "EXTRA_CONNECTION_ONLY", value = true)
43             putExtra(name = "EXTRA_CLOSE_ON_CONNECT", value = true)
44             putExtra(name = "android.bluetooth.devicepicker.extra.FILTER_TYPE", value = 1)
45         }
46         startActivity(intent)
47     }
48 }
```

```
AndroidManifest.xml MainActivity.kt x
14 class MainActivity : Activity() {
49     private fun solicitarConexaoBluetooth() {
51         val intent = Intent(action = Settings.ACTION_BLUETOOTH_SETTINGS).apply {
56             }
57         startActivity(intent)
58     }
59
1 Usage
60     private fun registrarCallbackDeAudio() {
61         AudioManager.registerAudioDeviceCallback(object : AudioDeviceCallback() {
62             override fun onAudioDevicesAdded(addedDevices: Array<out AudioDeviceInfo>?) {
63                 super.onAudioDevicesAdded(addedDevices)
64                 if (audioOutputAvailable(type = AudioDeviceInfo.TYPE_BLUETOOTH_A2DP)) {
65                     Toast.makeText(context = this@MainActivity, text = "Fone Bluetooth conectado!", duration = Toast.
66                 }
67             }
68
69             override fun onAudioDevicesRemoved(removedDevices: Array<out AudioDeviceInfo>?) {
70                 super.onAudioDevicesRemoved(removedDevices)
71                 if (!audioOutputAvailable(type = AudioDeviceInfo.TYPE_BLUETOOTH_A2DP)) {
72                     Toast.makeText(context = this@MainActivity, text = "Fone Bluetooth desconectado.", duration = Toast.
73                 }
74             }
75         }, handler = null)
76     }
77 }
```

8.5 Execução: Aplicativo forçando pareamento Bluetooth



9. Conclusão

O desenvolvimento desta prática possibilitou aplicar conceitos vitais na construção de softwares para dispositivos vestíveis (wearables). A interação direta com o hardware de áudio, tratada via AudioManager e com respostas automáticas através de intents de sistema, demonstrou ser uma abordagem eficiente para criar soluções que prezem pela acessibilidade. O resultado atende ao caso de uso da empresa "Doma", garantindo que os usuários tenham o meio físico adequado (fones Bluetooth) ativado para receber feedbacks auditivos de segurança e operação no ambiente de trabalho.

10. Referências

Não foram utilizadas referências bibliográficas para a elaboração das atividades, conforme as instruções do material de orientação.

